

Post-Disaster Rescue Guidance via VLM Fine-Tuning

CSE499A – Section 15, Group 5, Update 4

Name	Student ID
Abdullah Al Noman	2022095042
Tamanna Akter Mou	2211951042
Aryan Sami	2231407042
Ridita Afrin Riya	2211622042
Abrar Mohammed Tanzim Alam	2222864042

Work Done by Abrar Mohammed Tanzim Alam

My primary objective this week was to complete the reproduction of fine-tuning and evaluation pipeline of the DisasterM3 (DM3) paper (Wang et al., NeurIPS 2025, arXiv:2505.21089v2), whose released repository (Junjue-Wang/DisasterM3) provides only benchmarking code—no training scripts. This required self-assembling the full training, merging, and evaluation infrastructure from scratch on Kaggle’s single-T4 hardware, matching the hyperparameters in Appendix B.3 (LoRA rank = 64, $\alpha = 16$, dropout = 0.05, AdamW lr = 2×10^{-4} , cosine schedule, global batch 256, 1 epoch, vision encoder frozen). This week’s milestones focused on finalizing the training and deploying the evaluation infrastructure:

1. **Completed QLoRA Fine-Tuning (train_qwen_disasterm3.ipynb):** Successfully executed the 11th and final checkpoint-resume session on Kaggle’s 12-hour constrained hardware, reaching 100% completion (217/217 steps) of the 1-epoch fine-tuning schedule. The final QLoRA adapter weights were securely backed up to the Hugging Face Hub (AbrarAlam/disasterm3-qwen2.5v17b-qlora).
2. **Model Merging and FP16 Precision Push (1_merge_and_push_model.ipynb):** Engineered an isolated session environment to merge the QLoRA adapter with the base Qwen2.5-VL-7B model, actively circumventing numpy/scipy ABI mismatch crashes. The final fused 15GB checkpoint was successfully deployed to Hugging Face (AbrarAlam/disasterm3-qwen2.5v17b-mergedFP) in native FP16 precision.
3. **vLLM Evaluation Pipeline Engineering (3_vllm_evaluation.ipynb):** Developed and executed the evaluation notebook to run the paper’s benchmark script on the 6 MCQ tasks from Table 2. Overcame severe environmental blockers (T4 FlashAttention incompatibility and bitsandbytes/triton conflicts) by hot-wiring the vLLM engine to utilize **GPU T4 2× Tensor Parallelism (tensor_parallel_size=2)** with eager execution (**enforce_eager=True**).
4. **Automated Evaluation Execution (3_vllm_evaluation.ipynb):** Dynamically patched the original authors’ evaluation script at runtime to eradicate Windows-style pathing `FileNotFoundError` crashes and initialized the evaluation loop across the 6 target MCQ tasks. Segmented the massive 12-hour evaluation workload into dual 6-hour Kaggle sessions to ensure data persistence against kernel timeouts.

Work Done by Tamanna Akter Mou

1. **Pivot to the Satellite Imagery Pipeline:** Shifted implementation work to the satellite-imagery component because the initially selected paper did not provide open-source code. Began adapting the xBD-S12 repository of Dietrich et al. so its Sentinel-based preprocessing workflow could be applied to an alternative disaster dataset.
2. **Pipeline Troubleshooting and Code Review:** Investigated failed execution attempts caused by complex environment dependencies and read-only directory restrictions. Collaborated with Aryan to trace the repository’s data flow, configuration requirements, preprocessing stages, and expected intermediate outputs.
3. **Implementation Mechanics Documentation:** Mapped the Copernicus Sentinel-1 and Sentinel-2 processing pipeline in detail. Documented how building footprints are used as Ground Control Points (GCPs) to correct image georeferencing and how the aligned Sentinel patches are resampled to a uniform 128×128 resolution using Lanczos interpolation, providing a clear basis for adapting the pipeline to the new dataset.

Work Done by Abdullah Al Noman

1. **Super-Resolution Implementation:** Implemented the super-resolution model (from last week’s presented paper, *Shakya-From Pixels to Semantics*) to enhance post-disaster AIDER dataset imagery. To satisfy the Video Restoration Transformer’s (VRT) temporal dependency constraints, I built a custom script (**prep_vrt_input.py**) that structured a REDS-compliant directory (`/testsets/custom_disaster/LQ/000`) and replicated a static 1024×1024 frame four times (`0000.png--0003.png`) to generate the required 5D tensor format $[B, T, C, H, W] = [1, 4, 3, 1024, 1024]$.
2. **Inference and Memory Optimization:** The massive computational overhead of generating a 4096×4096 output array was mitigated by partitioning the 1024px input space into small, sequential spatial tiles of 256×256 pixels. The runtime command locked the temporal processing slice to 4 frames (`--tile 40 256 256`) and implemented a strict 20-pixel overlapping boundary padding (`--tile_overlap 2 20 20`) to insulate the data loader from threading instability while managing the free-tier GPU constraints.
3. **Execution, Fusion, and Visual Evaluation:** Finally, Fused overlapping tile margins via a recurrent blending utility to eliminate grid seams, successfully generating a crisp 4096×4096 asset at `/results/.../000/0000.png`. Furthermore, I wrote a verification script (**test.py**) utilizing Matplotlib to dynamically extract and contrast 100px center crops from the original and upscaled assets side-by-side, confirming the successful recovery of sharp micro-structural textures over pixelated blur.

Work Done by Aryan Sami

1. **Label Conversion Script (json_to_mask.py):** Developed a preprocessing script to convert xView2 (xBD) polygon annotations into pixel-level masks. The script parses only post-disaster JSON files, extracts building polygons and their damage labels (no damage, minor damage, major damage, and destroyed), maps them to integer class values from 0 to 4, and uses OpenCV’s `fillPoly` function to draw the polygons on blank 1024×1024 canvases. The resulting PNG masks are saved in a dedicated `masks_png` folder and are ready for model training.

2. **Dataset Loading Pipeline (`data_loader.py`):** Implemented `OriginalXBDDataset`, a custom PyTorch `Dataset` that pairs post-disaster PNG images with their corresponding masks. Images are normalized from the 0–255 range to 0.0–1.0, converted to tensors, and reordered to `[Channels, Height, Width]`. Following Section 2.1.3 of the paper, labels are simplified into three classes: background (0), intact/no damage (1), and damaged/minor, major, or destroyed (2). The data loader supplies correctly paired batches of four samples during training.
3. **Model Architecture (`model.py`):** Implemented a U-Net semantic-segmentation model with an ImageNet-pretrained ResNet34 backbone. In accordance with the paper’s ablation findings, only `layer1`, `layer2`, and `layer3` are used in the encoder to reduce disaster-specific overfitting. A decoder composed of `ConvTranspose2d` layers progressively restores spatial resolution, while a final 1×1 convolution produces three output channels for background, intact, and damaged classes. Bilinear interpolation ensures that the output damage map exactly matches the 1024×1024 input resolution.
4. **Training Pipeline (`train.py`):** Integrated the dataset loader and segmentation model into a complete training pipeline with automatic CUDA detection and CPU fallback. Training uses pixel-wise `CrossEntropyLoss`, the AdamW optimizer with a learning rate of 5×10^{-4} and weight decay of 1×10^{-4} , and a `CosineAnnealingLR` scheduler over 40 epochs. For every batch, the pipeline performs prediction, loss calculation, backpropagation, and model-weight updates using real xBD satellite image–mask pairs.

Work Done by Ridita Afrin Riya

1. **Local Environment Setup and Model Optimization:** Installed the Ollama framework locally to enable running large Vision-Language Models (VLMs) offline for image analysis. Transitioned to the LLaVA (Large Language-and-Vision Assistant) model to mitigate significant performance bottlenecks and high inference times caused by local hardware constraints when processing image pairs.
2. **Pipeline Codebase Synchronization and Debugging:** Synchronized the local workspace with the remote repository by pulling the latest changes to ensure the implementation utilized the most up-to-date scripts. Investigated the `VLM_Damage_Assessment.py` script and iteratively modified the codebase to process smaller data subsets (scaling from the hardcoded 227 pairs down to 10, 5, and finally 2 data points) to accommodate hardware limitations and facilitate rapid testing.
3. **Damage Assessment Pipeline Execution:** Executed the pipeline flow to load `pre_disaster.png` and `post_disaster.png` image pairs from folders within the `./data/images/` directory. Prompted the local LLaVA model to act as a damage assessment expert, categorizing structural damage on a 1–4 scale and streaming an analysis of building conditions and potential hazards.
4. **Output Generation and Formatting:** Formatted the model’s analysis to save dynamically as detailed `.txt` reports into the `./results/vlm_assessments/llava/` directory. Applied a strict naming convention corresponding to the specific data point (e.g., `00000000_assessment.txt`).

***Next Week’s Plan:** Evaluate strategies to mitigate high model running times—such as batching, resizing images, or utilizing cloud compute if local hardware remains insufficient—and scale up the analysis to process the full dataset once performance is properly optimized.